

Návrh a implemetace databázových systémů

Procedurální rozšíření jazyka SQL,
triggery, další podrobnosti

Triggery

- trigger
 - jakákoli procedurální funkce (ne SQL funkce)
 - volá se při určité události v DB
 - lze volat
 - před nebo po příkazu INSERT, UPDATE, DELETE
 - jednou pro každý modifikovaný řádek (*row-level*)
 - jednou pro každý SQL příkaz (*statement-level*)
 - nemůže sahat na jednotlivé řádky
 - funkce nesmí mít žádné parametry a musí vracet datový typ **trigger**
 - nejprve definujeme příslušnou funkci, pak ji teprve učiníme triggerem: **CREATE TRIGGER**

Triggery

- before-trigger
 - spouští se PŘED asociovaným příkazem
 - statement-level
 - spustí se těsně před vyvoláním asociovaného SQL příkazu
 - row-level
 - spustí se těsně před tím, než se začne operovat s daným řádkem

Triggery

- after-trigger
 - spouští se PO asociovaném příkazu
 - statement-level
 - spustí se na konci vykonávání asociovaného SQL příkazu
 - row-level
 - spustí se na konci vykonávání asociovaného SQL příkazu, ale předtím, než se spouští statement-level trigger

Triggery

- návratové hodnoty
 - statement-level trigger
 - příslušná funkce by měla vždy vracet NULL
 - row-level trigger
 - mohou vracet NULL nebo řádek tabulky
 - before-trigger
 - NULL: operace s daným řádkem se vůbec neprovede
 - pro příkazy INSERT a UPDATE bude vložen ten řádek, který je vrácen triggerem
 - trigger může ovlivnit, co se vloží a co se bude měnit
 - pokud nechceme ani jednu z těchto dvou možností, je nutno pečlivě dbát na to, aby trigger vracel přesně ten samý řádek, na němž se bude operovat
 - after-trigger – je jedno co vrací

Triggery

- row-before-trigger
 - typicky pro kontrolu IO, konzistence vkládaných dat, vložení aktuálního času apod.
 - není vhodný pro kontrolu referenční integrity
 - before-trigger nemůže s jistotou vědět, jak příkaz dopadne
- row-after-trigger
 - zápisy aktualizovaných hodnot do vázaných tabulek, kontrola konzistence

Triggery

- funkce triggeru může spouštět SQL příkazy, na nichž visí další trigger, a tak donekonečna, může se to volat i rekurzivně
- ilustrativní příklad

```
CREATE FUNCTION emp_stamp() RETURNS trigger AS $$  
BEGIN
```

```
    ...
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER emp_stamp  
    BEFORE INSERT OR UPDATE ON emp  
    FOR EACH ROW EXECUTE PROCEDURE emp_stamp();
```

Triggery

■ deklarace

```
CREATE TRIGGER name { BEFORE | AFTER } { event [ OR ... ] }  
    ON table  
    [ FOR EACH { ROW | STATEMENT } ]  
    EXECUTE PROCEDURE funcname (arguments);
```


Triggerové funkce

- trigger je konstrukce, která přiřadí nějaké události v databázi nějakou obslužnou funkci
 - obecně jedna obslužná funkce může být přiřazena více triggerům
- triggerové funkce
 - psané v procedurálním jazyce, např. PL/pgSQL
 - nesmějí mít vstupní parametry
 - parametry se předávají jinak (viz dále)
 - musí vracet jednu hodnotu datového typu *trigger*

Triggerové funkce

- když je funkce zavolána, automaticky se vytvoří následující proměnné
 - **NEW**
 - datový typ record
 - pro INSERT/UPDATE v row-level triggerech uchovává nově vkládaný řádek
 - pro statement-level trigger a DELETE je NULL
 - **OLD**
 - datový typ record
 - pro DELETE/UPDATE v row-level triggerech uchovává původní či odstraňovaný řádek
 - pro statement-level trigger a INSERT je NULL

Triggerové funkce

- když je funkce zavolána, automaticky se vytvoří následující proměnné
 - **TG_NAME**
 - datový typ name
 - jméno právě spuštěného triggeru
 - **TG_WHEN**
 - datový typ text
 - AFTER nebo BEFORE, podle typu triggeru
 - **TG_LEVEL**
 - datový typ text
 - STATEMENT nebo ROW, podle typu triggeru

Triggerové funkce

- když je funkce zavolána, automaticky se vytvoří následující proměnné
 - **TG_NARGS**
 - datový typ integer
 - počet parametrů, definovaných v triggeru
 - **TG_ARGV[]**
 - datový typ array of text
 - jednotlivé parametry z deklarace triggeru, index od 0
- ... další proměnné s informacemi, na jaké tabulce operujeme (**TG_TABLE_NAME**), v jakém schematu (**TG_TABLE_SCHEMA**), na jakém objektu (**TG_RELID**) a jakou operaci (**TG_OP**) provádíme

Triggerové funkce

- triggerová funkce
 - musí vracet
 - buď NULL
 - nebo přesně takový řádek, jaký odpovídá tabulce, na níž byl trigger spuštěn
 - lze tedy modifikovat proměnnou NEW a upravit data, která chceme změnit
 - může ukončit celou transakci tím, že vyvolá chybu

Triggerové funkce

- příklad
 - kdykoli se něco děje na tabulce zaměstnanců, je o tom učiněn záznam do auditové tabulky

```
CREATE TABLE emp (  
    empname text NOT NULL,  
    salary integer );
```

```
CREATE TABLE emp_audit (  
    operation char(1) NOT NULL,  
    stamp timestamp NOT NULL,  
    userid text NOT NULL,  
    empname text NOT NULL,  
    salary integer );
```

Triggerové funkce

```
CREATE FUNCTION process_emp_audit() RETURNS TRIGGER AS $$ BEGIN
    IF (TG_OP = 'DELETE') THEN
        INSERT INTO emp_audit SELECT 'D', now(), user, OLD.*;
        RETURN OLD;
    ELSIF (TG_OP = 'UPDATE') THEN
        INSERT INTO emp_audit SELECT 'U', now(), user, NEW.*;
        RETURN NEW;
    ELSIF (TG_OP = 'INSERT') THEN
        INSERT INTO emp_audit SELECT 'I', now(), user, NEW.*;
        RETURN NEW;
    END IF;
    RETURN NULL;
END; $$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER emp_audit
    AFTER INSERT OR UPDATE OR DELETE ON emp
    FOR EACH ROW EXECUTE PROCEDURE process_emp_audit();
```

Triggerové funkce

- odstranění triggeru

```
DROP TRIGGER emp_audit ON emp;
```


PL/pgSQL pod pokličkou

- je třeba znát některé detaily fungování
- náhrada parametrů
 - i pojmenované parametry funkcí se vnitřně přejmenovávají na očíslované (\$n)
 - pokud se uvnitř funkce vyskytne identifikátor, který je stejný jako některý z parametrů, je převeden na číslo parametru
 - tudíž není vhodné pojmenovávat stejně
 - parametry
 - lokální proměnné
 - tabulky a jejich sloupce

PL/pgSQL pod pokličkou

- plan caching
 - pro provádění se nejprve udělá plán a ten se pak používá znovu a znovu
 - pokud je v plánu nějaký SQL příkaz, naplánuje se včetně OID (identifikátorů objektů)
 - pokud mezitím změníme odkazovaný objekt např. tím, že provedeme DROP a pak zase CREATE, plán havaruje
 - řešení: používat CREATE OR REPLACE, což nemění OID

PL/pgSQL pod pokličkou

- plan caching

```
CREATE FUNCTION populate() RETURNS integer AS $$  
DECLARE  
    -- declarations  
BEGIN  
    PERFORM my_function();  
END; $$ LANGUAGE plpgsql;
```

- jestliže DROPneme a znovu vytvoříme funkci my_function(), funkce populate() ji nenajde, protože v příkazu PERFORM předpokládá určité OID, které je znovuvytvořením změněno